WASP Software Engineering Assignment

Ziyuan Wang

June 2026

1 Introduction

My research focuses on Tabular Foundation Models(TFMs), a popular area that aims to develop models capable of generalizing across diverse datasets without dataset-specific training. Unlike images or natural language, tabular datasets are much more structured but with abundant feature spaces, statistical properties, and semantic meanings, making obtaining generalizable TFMs particularly challenging.

However, current models like TabPFN[1][2] or TabICL[3][4] have bridged this gap. They are in-context learners and can predict any target cell given rows. They both show the strong power that deep learning methods can outperform the tree-based models, which was dominant in the past years[5].

A major line of my recent research are hypernetwork-based approaches to learn transferable tree-structured tabular foundation models. Hypernetwork is a new paradigm that tries to generate weights for the downstream models. The success of hypernetwork for MLP-like model, Mothernet[6], arouses the interest in exploring if it can benefit tree-structured models, to make the predictions more interpretable. But one barrier to overcome is that original decision tree is not differentiable and hard to train via gradient descent. Several literature tried to propose differentiable tree models, such as GradTree[7] and DNDT[8].

To summarize, my research is algorithm-oriented, aiming to explore TFMs in a more reliable, interpretable and generalizable way.

2 Lecture Principles

During the lectures, the discussion part is quite intriguing and relevant to my own research.

2.1 Discussion Question 1

In my own research project, what is the most important artifact to test or validate: code, data, model, prompt, environment, pipeline, evaluation metric, deployment, or something else? What could silently break in your project without an exception or obvious error message? What is one type of testing that you could add to your own project? Why that one? From my perspective, I consider the evaluation pipeline to be the most important component to validate. My research focuses on TFMs, whose performance is typically assessed using large benchmark suites such as OpenML[9] or TabArena[10]. Models are expected to generalize across dozens of heterogeneous datasets. Consequently, the correctness of the evaluation pipeline is as important as the model itself.

One aspect that could silently break without producing any exception is the benchmark evaluation process. For example, an unintended modification to data preprocessing, feature encoding, train-validation-test splitting, context construction for in-context learning, or the computation of evaluation metrics could change the reported benchmark results while the entire training and evaluation pipeline still executes successfully. Since the final performance is usually reported as an average ranking or average score across many datasets, these silent errors may not be immediately noticeable but can lead to incorrect conclusions about whether one model truly outperforms another. To improve the reliability of my research, I would have a small but representative subset of benchmark datasets for testing. Such tests would quickly detect unintended changes in preprocessing, evaluation logic, or data handling before launching expensive large-scale benchmark experiments. This would improve the reliability of the reported results.

2.2 Discussion Question 2

In your view, is testing in AI/ML important or secondary to other activities such as data collection, theory development, modeling, experimentation, publication, or deployment? Which activities matter most, and why? In my view, testing is an essential component of AI/ML research, but it should not be isolated from other activities. For my own research on tabular foundation models, I consider benchmark construction and evaluation design to be the most important activities. These models are intended to generalize across many datasets, and their success is typically measured by aggregated benchmark performance rather than

results on a single dataset. Consequently, if the benchmark contains data leakage, inconsistent preprocessing, or an unfair evaluation protocol, even perfectly tested code may produce misleading conclusions. In other words, testing ensures that the experiments are executed correctly, whereas a well-designed benchmark ensures that the experiments answer the correct scientific question.

3 Guest Lecture Principles

3.1 What will happen to software engineers with AI coming — Change becomes a persistent condition

In AI-driven software development, change is no longer an occasional event but a persistent condition. Engineers are required to continuously adapt to evolving tools, models, and workflows rather than transitioning between stable phases. For example, many companies require the employees to use agentic tools such as OpenClaw or Claude Code. This shifts the focus from one-time system design to continuous learning. If in the upcoming years new tools are invented, people should embrace them frequently.

3.2 What will happen to software engineers with AI coming — Less routine work is not automatically better work

Although AI systems can automate many repetitive and monotonous tasks, this does not necessarily reduce the overall loads of engineering work. Instead, effort is shifted toward responsibilities such as system design, validation of AI-generated outputs, and coordination of complex pipelines, which could be even harder or more time-consuming, requiring careful management of focus, task switching, and recovery to maintain sustainable productivity.

4 Data scientists, software engineers, and AI engineers

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not? Data scientists have backgrounds in statistics and ML algorithms, focus on building accurate models, use exploratory workflows (e.g., Jupyter notebooks), and evaluate primarily on test accuracy. Software engineers focus on delivering complete products, design system architecture,

implement, test, deploy at scale, and navigate trade-offs between usability, scalability, security, and cost. Personally, I do not fully agree with a strict separation between data scientists and software engineers. In practice, tasks such as data preparation, model implementation, training execution, and evaluation are all parts of modern machine learning workflows and are often carried out by data scientists themselves. From this perspective, I view a data scientist as a software engineer with additional expertise in mathematics, statistics, and machine learning.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area? For the evaluation of TFMs, they are often evaluated given a certain computational budgets, which means that we can't design a really complicated models to achieve marginal improvement. In addition, tabular benchmarks include strong classical baselines, which are highly optimized and remain competitive. Therefore, the overall performance is not solely determined by model design, but also by how the evaluation system is constructed, including dataset selection, preprocessing pipelines, hyperparameter tuning budgets, and computational constraints. This highlights that evaluating TFMs is a system-level problem rather than a purely model-centric one.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain. I don't think so. Because I think AI bridges the gap between these two roles. From one side, AI agent can perform well on model implementation, experiment design and evaluation. Software engineers can conduct ML tasks with its help conveniently. From the other side, AI agent can also perform requirement analysis, model deployment and testing. Data scientist also have the relevant knowledge about software engineering as well, which were called 'Unicorn' in the given book. I think these people are more and more common in the future.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct?

The term "AI Engineering" most commonly refers to building automated pipelines, deploying models, and MLOps. In this sense, I think AI Engineering is primarily an extension of software engineering applied to AI systems, with additional considerations related to data dependency, model uncertainty, and continuous retraining.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries

between these roles?

Foundation models can make software engineers available to SOTA AI models whereas data scientists might become conducting engineering tasks such as prompt engineering. AI coding assistants can decrease coding barriers for both roles. RAG systems require on both collaboration between systems and models, which need both two roles work together. Agentic workflows is the ultimate systems integration problem, requiring both roles to work closely on model reasoning, tool development and agent deployment.

5 Paper Analysis

5.1 Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation

5.1.1 Core ideas and their SE importance

Most of the benchmarks for agent evaluation are based on GitHub issues rather than reflect how people chat with coding assistants in integrated development environments (IDEs). This paper uses mutation methods to transform formal GitHub issue descriptions into realistic user-style queries based on telemetry analysis of a popular chat-based agent interactions[11]. The result reveals that agents are overestimated by existing benchmarks after mutation. As for the importance of the paper, it provides a new method to benchmark the agent using the mutation, which can imitate the real-life scenario when people interact with agents.

5.1.2 Relation to my research

Current benchmarks such as TabArena[10] or TabZilla[12] for TFMs often present tables without metadata directly. However, real-world tabular data is rarely used without surrounding metadata: column descriptions, data provenance, business semantics, unit information, or known data quality issues. In practice, human analysts constantly rely on such metadata to make sense of tables, yet most dominant model evaluations intentionally ignore it. Even for some models such as ContextTab[13] and TabSTAR[14] with metadata as an input, the effect of the metadata still seems to be minimal.

The problem that kicks in is that the presence or absence of metadata can substantially change the predictions of the models, but the effect is context-dependent. For some tasks (e.g., simple imputation of obvious missing values), metadata may not matter. For others (e.g., deciding how to encode a

categorical variable, interpreting a column named "CODE" without further explanation), metadata could be decisive. This coincides with the SWE-Bench findings: just as code agents fail on terse user queries, TFMs may be underestimated when metadata is absent.

5.1.3 Integration into a larger AI-intensive project

This work fits into a broader research agenda on reliable AI for data-centric tasks. While SWE-Bench mutations target code generation, the mutated idea can be applied to any other tasks such as long-context understanding and step-by-step solution to mathematical problems. If many other benchmarks are turned into a user-style queries based on the conversational interaction, benchmarking for the LLMs could be more fair and easier to compare among different models.

5.1.4 Adaptation of my research

Directly applying SWE-Bench mutations to Tabular FMs is infeasible—the domains differ (code vs. tabular data) and mutation targets differ (natural language queries vs. metadata). However, the core idea meets each other: systematically alter task presentation to reveal model sensitivities. Specifically, I plan to create multiple evaluation versions of existing TFMs, which support metadata. This will allow us to measure performance variance by altering the metadata, identify predictions from the models, how much it will depend on the metadata. The goal is not to propose a new benchmark, but to test the role of meta data play when conducting tabular predictions.

5.2 Prompting in Practice: Investigating Software Developers' Use of Generative AI Tools

5.2.1 Core ideas and their SE importance

The paper investigates how professional software engineers integrate generative AI tools into their daily development workflow through a survey-based empirical study. The survey includes GenAI usage patterns, prompting and conversation strategies and GenAI reliability and common issues[15]. One of the main findings is that prompting is an iterative engineering activity instead of a one-time interaction. Developers continuously refine prompts, verify generated outputs, and combine AI assistance with their own expertise. Furthermore, the study highlights that AI tools are most effective when integrated into existing software engineering workflows rather

than replacing engineers.These findings are important for AI engineering because they emphasize that building AI-intensive systems requires more than high-performing models. Effective workflows, validation mechanisms, and human-AI collaboration become essential engineering concerns.

5.2.2 Relation to my research

The finding one from the paper is interesting, which shows that there is highly positive correlation between GenAI proficiency and the usage ranges, also the junior and intermediate users tend to rate themselves as more proficient in using AI. From my perspective, the simplest way to GenAI is to generate code, which is almost commonly used by every programmer. But GenAI is also powerful in refactoring, debugging or even testing. During my own research, I applied GenAI in multiple ranges and really cautious about code generation, which I think might lead to difficult readability. Compared with other juniors around me, they tend to use GenAI to do more abundant and fancy tasks than I did, which has similar results as the paper suggested.

5.2.3 Integration into a larger AI-intensive project

A suitable AI-intensive software project could be an intelligent tabular data analysis platform, such as trading system. The system would assist users to trade stocks or options, because most of the information are stocked by tables.

The ideas that there are different text-formatting strategy from the paper could improve the trading strategies. Instead of simply predict the price in the next day. Using different tabular information and sending them to GenAI can help to provide more comprehensive understanding of the market. For my current WASP research, we are still in the very beginning stage of tabular foudnation models. We only focused on value predictions instead of integration with LLM to do generation. But I think there should be multi-modal models like vision-language models.

5.2.4 Adaptation of my research

This paper encourages me to think how I can adapt my current research with GenAI. I am somewhat cautious about GenAI but I need to find a sensible way to use it. Recently, I have gone through many repositories of many tabular foundation models, each of which has huge code base, which is hard to read alone. With the help of AI, I not only let it run the code for me, most of the time I use it to analyze the code structure and teach me

the code details to me, which saved many time and vigor. Like the paper suggested, as I become more and more proficient when using AI, I can use it to handle many different tasks, which is a huge value to me.

6 Evidence, originality, and GenAI-use transparency

All the papers cited in the report, can be found in the reference section at the end. Generative AI was used to examine grammar mistake during the writing.

References

[1] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. Tabpfn: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848*, 2022.

[2] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025.

[3] Jingang Qu, David HolzmÃžller, GaÃl Varoquaux, and Marine Le Morvan. Tabicl: A tabular foundation model for in-context learning on large data. *arXiv preprint arXiv:2502.05564*, 2025.

[4] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. Tabiclv2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139*, 2026.

[5] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems*, 35:507–520, 2022.

[6] Andreas Müller, Carlo Curino, and Raghu Ramakrishnan. Mothernet: A foundational hypernetwork for tabular classification. *arXiv e-prints*, pages arXiv–2312, 2023.

[7] Sascha Marton, Stefan Lüdtke, Christian Bartelt, and Heiner Stuckenschmidt. Gradtree: Learning axis-aligned decision trees with gradient descent. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 14323–14331, 2024.

[8] Yongxin Yang, Irene Garcia Morillo, and Timothy M Hospedales. Deep neural decision trees. *arXiv preprint arXiv:1806.06988*, 2018.

[9] Bernd Bischl, Giuseppe Casalicchio, Matthias Feurer, Pieter Gijsbers, Frank Hutter, Michel Lang, Rafael G Mantovani, Jan N van Rijn, and Joaquin Vanschoren. Openml benchmarking suites. *arXiv preprint arXiv:1708.03731*, 2017.

[10] Nick Erickson, Lennart Purucker, Andrej Tschalzev, David Holzmüller, Prateek Desai, David Salinas, and Frank Hutter. Tabarena: A living benchmark for machine learning on tabular data. *Advances in Neural Information Processing Systems*, 38, 2026.

[11] Spandan Garg, Benjamin Steenhoek, and Yufan Huang. Saving swe-bench: A benchmark mutation approach for realistic agent evaluation. *arXiv preprint arXiv:2510.08996*, 2025.

[12] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? *Advances in neural information processing systems*, 36:76336–76369, 2023.

[13] Marco Spinaci, Marek Polewczyk, Maximilian Schambach, and Sam Thelin. Contexttab: A semantics-aware tabular in-context learner. *Advances in Neural Information Processing Systems*, 38:147005–147040, 2026.

[14] Alan Arazi, Eilam Shapira, and Roi Reichart. Tabstar: A tabular foundation model for tabular data with text fields. *Advances in Neural Information Processing Systems*, 38:172108–172161, 2026.

[15] Daniel Otten, Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, and Denys Poshyvanyk. Prompting in practice: Investigating software developers' use of generative ai tools. *arXiv preprint arXiv:2510.06000*, 2025.